DOCUMENTATION

DoomSquad is a simple shooter inspired by games like Doom and Wolfenstein, using a raycast-based rendering engine.

Startup:

Game can be started by simply calling cargo run

Features:

Movement/Controls:

- W/A/S/D: Forwards/Left/Right/Backwards
- Shift: Sprint
- Space: Jump; bigger with greater movement speed
- C: Crouch, or, when moving, Slide; sliding gives a small initial speed boost => cooldown to prevent abuse
- R: Rocket Jump: more powerful jump with cooldown, possible while airborne
- F: interact
- Arrows Up/Down: move Crosshair up/down
- Arrows Left/Right: turn left/right
- E: toggle Fine Aiming: reduces sensitivity of arrow key for easier aiming. Crosshair will change color.
- Right Control: Fire Bullet - little damage, no cooldown
- Right Shift: Fire Arrow - more damage, cooldown, affected by gravity
- G: Enter Godmode: no damage, no collisions with map, no gravity. In Godmode, long-pressing Space will caause you tto fly up, Shift wil cause you to fly down
- L: Lock all currently active enemies (including projectiles); player can still move; still able to receive damage when walking into bullets or enemies
- Escape: exit to main menu

Pressing C+Space+Shift+W/A/S/D simultaneously (within one frame) will result in a "Kamil Jump": larger than normal Jump, smaller than Rocket Jump. Sliding cooldown must be finished for this to work, as the jump is bigger because of the greater speed a slide provides.

Entities:

Entities are rendered to always face the player; represent something that is able to move and do things (as opposed to static map geometry); they are able to display animations when moving or performing an action (like attacking)

- Melee Enemy: looks a bit like a spider; attacks at close range with its legs
- Weak Enemy: looks like a blue box with two legs and a mouth; attacks at close range by biting the player
- Summoner: looks a bit like an octopus; periodically summons group of Weak Enemies
- Ranged enemy; a small tower with a red orb; fires bullets at the player; unable to move itself
- Barrel: explodes when shot, deals massive damage to anyone standing nearby
- Bullets (blue if fired by player, red if fired by enemy): travel in a straight line, deal damage on impact
- Arrows: deal damage like bullets, but can only be fired by player; deal more damage; are affected by gravity (traces of an archer enemy can be found in the code, but that is not implemented)
- Dummy: does nothing, just stands around; has quite a lot of health

Interactables

Rendered in the same way as entities; can be interacted with using F for various effects

- Map Switcher: has a map icon; loads the player onto a different map, but only if there is at most one entity left on the map
- Spawner: has a yellow warning icon, spawns an entity
- Heal Station: red heart icon; restores a small amount of health; onyl usable once
- Jump Pad (sometimes called Elevator in Code): propels the player upwards
- Slot Machine: randomly either restores some health, or spawns a melee enemy; Easteregg: small chance (10%) of swapping map to old test map that was used during developmen

Map Geometry

A map consists of two types of geometry, both with textured sides:

- Walls: meant to enclose a level; if a the renderers rays encounter a wall, anything behind it will not be rendered (=> performance); unless in Godmode, they completely block movement
- Blocks: meant to construct the details of a level; have a unicolored floor/ceiling (collectively called surface in the code); can be walked (or jumped...) onto, if not too high

Game Progress

Player will be spawned into a level; at the end of each level, there is an Interactable to swap to the next map; only pssible when there is at most one entitiy left in the whole level

Progress within a level can not be saved; when player dies or exits using Escape, they can choose to save their current level; next time the load from the main menu, they will be returned there (persistent even when closing game)

Simple explanation of Renderer

The map consists of "sides" belonging either to a block or a wall; for every column of the screen, a ray is sent out, which checks for intersection with each side. It does this by applying an affine linear transformation to the space so that the side is on the x axis if the ray intersects it (see image "intersectfunctionexplainer.png); this also makes determining the distance, and how far along the side the intersection was, easy.

The rayhits that were found are then grouped according the blocks they belong to (creating block slices), from which Renderer Tasks are created, which can then be drawn onto the screen. RendererTasks for entities and sides of blocks/walls include a texture, which must be scaled correctly. Tasks representing a surface do not have a texture and render a unicolor.

Note: The collision system uses an essentially identical intersection function to see if the player walked into a wall; if a ray is cast in one direction and has an odd number of intersections with sides of one block, we are inside that block in 2d level space; this principle is used by the collider for collision with blocks.

Parsers

Maps, with the entities and interactables in them, can be created in GeoGebra, in an appropriate format (for example, the value of the red cahnnel of the RGB value of a polygons color in GeoGebra corresponds to how high the bottom of that block is). An parser system exists to convert such GeoGebra files into ingame maps.

TESTING PHILOSOPHY

Main method was "testing-by-doing"; given that this is a game, with lots of diffeent states, which are dependent on one another, we felt that this was sensible.

Some smaller/simpler functions have direct tests added for the sake of completeness, although their correct working is apparent when simply playing the game, too.

Tests can be found at the bottom of the files of their respective functions.

AI USE DISCLAIMER

LLMs (ChatGPT, Gemini) were used at times for understanding and research (Google, Stackoverflow substitute, essentially); very few functions were generated by these models directly, as well as some tests; sometimes models were used for finding logic errors in large amounts of code. Overall, AI usage has probably been relatively minimal.